TOKENIZER · 从零讲透

BPE
分词器原理

一个只会数数的笨算法，
是怎么学会把人类语言切开的。

零基础 · 不预设任何前置知识



写给第一次接触分词器的人

全文 10 章 · 含完整手工演算 · 可复现代码

2026 · 11

目录

CONTENTS

00 先说人话：分词器到底在干什么

01 三个必须先钉死的概念

02 它是从哪来的：一段 30 年的历史

03 算法本体：只有 10 行的核心循环

04 手把手跑一遍：10 轮合并全过程

05 为什么"挑频率最高的"就对了

06 训练完之后：怎么用它来切句子

07 字节级 BPE：为什么永远不会"不认识"

08 BPE 与 WordPiece、Unigram 的区别

09 自己写一个：30 行 Python

10 十个常见误解

A 附录：术语表与一页速查

> **怎么读这份东西**
>
> 不用背。你只需要在读完第 4 章之后，能够合上文档，自己拿纸笔把"lowest"这个词的合并过程重演一遍。做到了，你就真的懂了；做不到，就回去再看一遍第 4 章。
>
> 全文所有数字都是真实跑出来的，代码在第 9 章，你可以自己验证。

CHAPTER 00

先说人话：分词器到底在干什么

> 在讲 BPE 之前，得先搞清楚它服务的对象是谁、卡在什么环节上。不然你会背算法却不知道为什么要有它。

0.1 模型是个"数字机器"，它不认字

大语言模型内部做的事情，说白了就是**一堆矩阵乘法**。矩阵里装的是数字，所以模型从头到尾只能处理数字。你给它一句"今天天气不错"，它根本看不见这些字——它看见的是一串整数，比如 `[791, 10432, 8877, 3120]`。

那么问题来了：**谁负责把"字"翻译成"数字"？**

答案就是分词器（tokenizer）。它是模型外面的一层独立组件，不参与训练时的梯度更新，只做纯粹的字符串到整数的查表翻译。它做两件事：



反过来，模型吐出一串 ID，也要靠分词器翻译回文字，你才能看懂。所以分词器是**一座双向的桥**：encode 是过去，decode 是回来。

> **一句话记住**
>
> 分词器不聪明，它不会"理解"语言。它本质上只是一本**切分手册 + 一张对照表**。BPE 就是用来生成这本手册的算法。

0.2 那为什么不干脆一个字一个字地切？

这是最自然的想法：英文 26 个字母，中文几千个常用字，直接按最小单位切，词表小得可怜，还永远不会遇到"不认识的字"。听起来完美。

但它有个致命问题：**句子会变得特别长。**

"hello" 变成 5 个 token，"机器学习" 变成 4 个 token。而 Transformer 的计算量是**跟序列长度的平方成正比**的——长度翻倍，算力开销变四倍。上下文窗口就那么大，全用来装字母了，模型能看到的信息反而变少了。

那按"词"切呢？整词切，一句话只要几个 token，效率高了。但另一个问题炸了：**词表会爆炸。**英语常用词几十万，加上人名、地名、新造词、拼写错误、代码符号、emoji……你永远列不完。更糟的是，一旦遇到词表里没有的词（这叫 **OOV，Out-of-Vocabulary**），你只能显示一个 `<UNK>`——信息直接丢了。

按字符切 Character	`l` `o` `w` `e` `s` `t` 词表极小、绝不 OOV，但序列长 6 倍，语义太碎
按整词切 Word	`lowest` 序列最短，但词表爆炸；遇到 "lowestest" 或 "lowesst" 直接变 <UNK>
按子词切 Subword · BPE	`low` `est` 常见词整块保留，罕见词自动拆成认识的碎片，词表可控

所以真实世界里选的是第三条路：**子词（subword）**。核心思想特别朴素——

> **常见的整块留着，罕见的拆开拼。**
> **永远不会遇到不认识的，也永远不会太长。**

打个比方。你有一盒乐高：按字符切，等于每块都是单个小凸点，什么都能拼但拼一面墙要一万块；按整词切，等于每个模型都是出厂粘死的一整块预制板，看起来省事，可一旦缺一块就彻底没救。子词是什么？是**既有 2×4 的常用砖，也有单点小粒**——常用组合直接拿现成的，缺什么就用小粒临时拼。

0.3 BPE 就是"怎么造出这盒乐高"的配方

注意这句话的重点：BPE 是**造积木的配方**，不是拼积木的动作。

很多人第一次接触会混淆这两件事。你下载一个 LLaMA 模型，里面有个 `tokenizer.json`，那里面存的不是"怎么训练"，而是**训练好的结果**——一张清单，写着"第 1 步合并 ea，第 2 步合并 re……"。运行时它只是照着这张清单执行，压根不统计频率。

这个区分极其重要，第 6 章会专门展开。现在你只需要记住：BPE 有两个完全不同的阶段。

阶段	什么时候做	它在干嘛
训练 Training	一次性。造模型之前，在大语料上跑一遍，可能几小时。	统计、比较、合并，反复几万次，产出「词表 + 有序合并规则」。之后固化成文件。
推理 Encoding	每次你输入一句话都要跑，要求毫秒级。	照着手册切分。不统计、不比较、不做任何"学习"。

> **这里最容易翻车**
>
> 看到 BPE 的伪代码时，很多人以为"每次分词都要重新统计频率"。完全不是。训练阶段统计频率是为了**决定合并的先后顺序**；一旦顺序定了，顺序本身就是全部知识。运行时只需要这个顺序。

CHAPTER 01

三个必须先钉死的概念

> 词表、合并规则、顺序。这三个词在后面每一章都会出现，现在花三分钟把它们焊死在脑子里，后面会轻松十倍。

1.1 词表（Vocabulary）：一本带编号的积木清单

词表就是**所有合法 token 的集合**，每个 token 有一个唯一编号。编号本身没有含义，只是数组下标。

一个真实的词表长这样（示意）：

```
# vocab.json 的片段
{
  "l": 75,
  "o": 78,
  "w": 86,
  "lo": 1204,
  "low": 5757,
  "est": 3598,
  "est</w>": 7412,
  "lowest": 21509
}
```

词表大小是个要拍板的超参数。GPT-2 是 50257，GPT-4 系列大约 10 万量级，LLaMA 3 是 128256，Qwen 系列约 15 万。为什么差这么多？后面第 5 章会讲清楚这个权衡。

1.2 合并规则（Merges）：一张有顺序的配方表

这是 BPE 的灵魂，也是最容易被忽略的细节。**合并规则是一个有序列表**，不是集合。

```
merges = [
  # 序号    左       右
  (1,  "e",   "s"),      # e + s → es
  (2,  "es",  "t"),      # es + t → est
  (3,  "est", "</w>"),  # est + 词尾 → est</w>
  (4,  "l",   "o"),      # l + o → lo
  (5,  "lo",  "w"),      # lo + w → low
  ...
]
```

顺序为什么重要？因为这些规则之间有**依赖关系**。第 2 条要用到第 1 条造出来的 `es`；如果你先执行第 5 条，根本还没有 `lo` 这个 token，规则就空转了。

换个说法：规则编号 = 优先级。编号越小，越先尝试。这一点在第 6 章编码时会变成一个具体的实现细节，也是初学者写出"结果不对"的代码的首要原因。

1.3 词尾标记：为什么会有个怪东西 `</w>`

原始论文里，每个词的末尾会被加一个特殊符号 `</w>`（end of word）。看名字就知道：它标记"这个词到此结束"。

为什么需要它？因为如果没有，**词内的片段和词尾的片段就分不开了**。考虑 "est"：它既可能是 "newest" 的结尾，也可能是 "establish" 的开头。这两个 "est" 的语法角色完全不同，但它们的字符一模一样。

加上词尾标记之后：

词	不加标记	加了标记
newest	`n e w e s t`	`n e w e s t </w>`
establish	`e s t a b l i s h`	`e s t a b l i s h </w>`

这样一来，只有 "newest" 里那个 `est` 后面跟着 `</w>`，它们会合并成 `est</w>`；而 "establish" 里的 `est` 后面跟着 `a`，永远不会。于是"词尾后缀"和"词中片段"被彻底分开了。

> **现代实现的两种做法**
>
> **显式标记**：用 `</w>`（原论文）或 `Ġ`（GPT-2 表示前置空格）。
>
> **隐式前缀**：WordPiece 反过来做，词中片段加 `##` 前缀（`est` → `##est`），表示"我不是词首"。
>
> 最新的 BPE 实现（如 tiktoken、HuggingFace tokenizers）干脆把"这个词是不是句首/空格后"直接编进预分词阶段，不再需要标记符。原理相同，只是实现演进。

CHAPTER 02

它是从哪来的：一段 30 年的历史

> 知道来路，能帮你理解它为什么长成这样——尤其是"为什么这个算法看起来这么笨"。

BPE 全称 **Byte Pair Encoding**，直译"字节对编码"。名字听着很唬人，其实意思就是"把一对字节编码成一个新东西"。

1994：它是为了压缩文件，不是为了语言

Philip Gage 在《C Users Journal》上发了一篇很短的文章，提出一个压缩算法：找出数据里出现最频繁的那一对相邻字节，用一个新字节替换掉它们，然后重复。就这么简单。

为什么这能压缩？因为原来占两个位置的组合，现在只占一个位置。反复几十轮之后，高频的字节组合被压缩成短符号，文件就小了。整个算法不需要任何语言学知识，它只是在做**统计上的去重**。

2016：有人发现，压缩出来的东西像"词根"

Rico Sennrich、Barry Haddow、Alexandra Birch 三个人在写神经机器翻译（NMT）论文时，被 OOV 问题卡住了：翻译模型遇到词表外的词就废了。他们想了个办法——用 BPE 在语料上跑一遍，把词切成语素差不多的碎片。

结果让他们自己也吃了一惊：算法完全不懂英语，但合并出来的东西里有大量像模像样的词缀。

```
# 在英语语料上跑 BPE，排名靠前的合并对
e + s   →  es
es + t  →  est
l + o   →  lo
lo + w  →  low
e + r   →  er
...
# 这些恰好就是英语里最常见的屈折后缀和词根
```

道理其实不神秘：语言里高频共现的字符组合，往往就是有语法功能的单位。**统计频率间接地捕捉到了形态学。**这是全书最重要的一句直觉，第 5 章会再展开。

这篇论文（《Neural Machine Translation of Rare Words with Subword Units》）成了 BPE 进入 NLP 的起点，至今被引几万次。

2019 之后：从字符级走到字节级

GPT-2 做了一件关键改动：不再以"字符"为最小单位，而是以 **UTF-8 字节**为最小单位。基础词表固定成 256 个，从此世界上任何文字、任何符号、任何 emoji 都不可能不在词表里。第 7 章细讲。

今天的局面：GPT 系列用 tiktoken（字节级 BPE），LLaMA / Qwen / Mistral 用 SentencePiece，但里面的切分算法**仍然是 BPE**。它已经统治了近十年，没人换掉它，因为——它足够好，而且足够快。

CHAPTER 03

算法本体：只有 10 行的核心循环

> 先看完整的骨架，再一行一行拆。别急着看懂细节——先感受一下它有多简单。

3.1 训练循环

```
输入：一个语料库（很多句子）、目标词表大小 V
输出：初始词表、merges 列表

# ① 初始化
把每个词拆成单个字符，末尾补 </w>
统计每个字符在语料中出现的总次数 → 这就是初始词表

# ② 反复合并，直到词表够大
while 词表大小 < V:
    统计所有「相邻 token 对」在这个语料里出现的总次数
    找出次数最多的那一对 (a, b)                    # ← 唯一的智能
    把所有相邻的 a, b 合并成一个新 token "ab"
    把 (a, b) 追加到 merges 列表                    # ← 顺序很重要
    把新 token "ab" 加入词表

return 词表, merges
```

真的就这么点东西。整个 BPE 里唯一需要"判断"的地方，就是那句**找出次数最多的那一对**。没有梯度、没有损失函数、没有反向传播。它是纯统计的，确定性的（给定语料和顺序，结果永远一样）。

3.2 逐行拆解

为什么是"相邻对"，不是"字符组合"？

因为 BPE 只做**局部拼接**。它一次只看两个挨着的东西，把它们粘起来。它不会把 "a…b" 这种中间隔了东西的组合当成一个整体——那需要更复杂的结构，BPE 故意不做。

这也决定了它的能力上限：BPE 造出来的 token 永远是**原文的连续子串**。它不可能造出一个"语义上相关但字面上不相邻"的 token。所以 BPE 的 token 一定能被原样拼回原文（这个性质叫 lossless，无损），这是它比很多花哨方法更可靠的地方。

为什么要乘频率？

语料里同一个词可能出现几万次。你不必真的存几万份，只需要存"词 + 它的频次"，统计时把频次乘上去。这是标准做法，也是为什么 BPE 训练实际上跑得挺快。

为什么选"最大"?

因为每合并一对，就省下 **等于该对频次** 个 token。省得最多的当然优先。第 5 章会专门论证这一点，这里先记住结论：合并频率最高的一对，等于做当前最优的一步压缩。

循环多少次?

两个常见的停止条件：

- **目标词表大小**：比如"我要 5 万个 token"，那就跑 (50000 − 初始字符数) 轮。GPT-2 是 256 字节 + 50000 次合并 + 1 个特殊符号 = 50257，数字完全对得上。
- **最小频次阈值**：当最高频的那一对出现次数低于某个阈值（比如 2），就停。意思是"剩下的都不值得合并了"。

3.3 复杂度：朴素实现 vs 真实实现

朴素实现每一轮都要**把整个语料重新扫一遍**统计频率。如果语料有 N 个 token，要合并 M 轮，那就是 O(N·M)。N 可能是几十亿，M 可能是 5 万——这个数字大得离谱，跑不完。

真实的实现（比如 HuggingFace 的 `tokenizers`，C++ 写的）用了两个技巧：

问题	怎么解决
每次重扫全语料	只更新**被影响的位置**。合并 a+b 之后，只有紧挨着新 token 的邻居对会变，其余全部不变。用增量更新代替全量重算。
每次找最大值	用一个**优先队列（最大堆）**存所有对及其频次，取最大值 O(log n)。惰性删除：弹出的对如果频次已过期，就丢掉重取。

这些都只是工程优化，不改变算法本身。你理解原理时完全可以忽略它们。

> **动手之前的一个心理准备**
>
> 下面第 4 章我们会用一个只有 4 个词、共 16 个 token 的玩具语料，手工跑 10 轮。你会发现它慢、重复、无聊。**这就是对的。**BPE 的美感不在于聪明，而在于"用最笨的办法反复做，最后得到的东西居然很有结构"。

CHAPTER 04

手把手跑一遍：10 轮合并全过程

> 这一章是全书的核心。下面的每一个数字都是真实跑出来的，不是编的。请拿一张纸，跟着走一遍。

4.1 准备语料

我们用 Sennrich 论文里那个经典的小语料。只有 4 个词，每个词带出现次数：

词	出现次数	含义
low	5	单独的 low
lower	2	low + er（比较级）
newest	6	new + est（最高级）
widest	3	wide + est（最高级）

4.2 初始化：全部拆成单字，末尾加词尾标记

把每个词打散成字符，并在末尾补上 `</w>`。这是第 0 轮，也是算法的起点。

```
l o w </w> ×5
l o w e r </w> ×2
n e w e s t </w> ×6
w i d e s t </w> ×3
```

初始词表就是所有出现过的字符（含 `</w>`）：

```
词表 = { </w>, d, e, i, l, n, o, r, s, t, w }
大小 = 11
当前 token 总数 = 5×4 + 2×6 + 6×7 + 3×7 = 95
```

> **先理解这个 95 是怎么来的**
>
> "low</w>" 有 4 个 token，出现 5 次，贡献 20 个；"lower</w>" 6 个 ×2 = 12；"newest</w>" 7 个 ×6 = 42；"widest</w>" 7 个 ×3 = 21。加起来 95。
>
> **这个数字是我们全程的"记分牌"。**每合并一次它就会下降，最后一轮结束你回头看它降了多少，就明白 BPE 到底在干什么了。

4.3 开始合并

每一轮都做三件事：**数所有相邻对 → 挑最多的 → 合并。**下面把 10 轮全部列出来。

ROUND 01 token 总数 95 → 86

候选（按频次降序）：`e+s:9` `s+t:9` `t+</w>:9` `w+e:8` `l+o:7` `o+w:7`

e + s → es 频次 9，三个并列第一，按字母序取第一个

语料变成：`l o w </w>` | `l o w e r </w>` | `n e w es t </w>` | `w i d es t </w>`
"newest" 和 "widest" 里都出现了 e 后面跟 s，一共 6+3=9 次。合并后少掉 9 个 token。

ROUND 02 token 总数 86 → 77

候选：`es+t:9` `t+</w>:9` `l+o:7` `o+w:7` `e+w:6` `n+e:6`

es + t → est 频次 9

注意：上一轮的输出 `es` 立刻成了这一轮的输入。这就是规则之间有**依赖**的意思。
语料：`l o w </w>` | `l o w e r </w>` | `n e w est </w>` | `w i d est </w>`

ROUND 03 token 总数 77 → 68

候选：`est+</w>:9` `l+o:7` `o+w:7` `e+w:6` `n+e:6` `w+est:6`

est + </w> → est</w> 频次 9

这一轮是全书最该停下来看三秒的地方。合并之后，`est` 和 `est</w>` 成了两个不同的 token。
语料：`l o w </w>` | `l o w e r </w>` | `n e w est</w>` | `w i d est</w>`
从此以后，"词尾的 est"（最高级后缀）和"词中的 est"（比如 establish 里那个）会被永远分开处理。词尾标记的意义就在这里。

ROUND 04 token 总数 68 → 61

候选：`l+o:7` `o+w:7` `e+w:6` `n+e:6` `w+est</w>:6` `w+</w>:5`

l + o → lo 频次 7（low 出现 5 次 + lower 出现 2 次）

语料：`lo w </w>` | `lo w e r </w>` | `n e w est</w>` | `w i d est</w>`

ROUND 05 token 总数 61 → 54

候选：`lo+w:7` `e+w:6` `n+e:6` `w+est</w>:6` `w+</w>:5`

lo + w → low 频次 7

语料：`low </w>` | `low e r </w>` | `n e w est</w>` | `w i d est</w>`
到这里，"low" 已经成了一个完整的 token。它不再是由 l/o/w 三个零件拼的，而是一整块积木。

ROUND 06 token 总数 54 → 48

候选：`e+w:6` `n+e:6` `w+est</w>:6` `low+</w>:5` `d+est</w>:3`

e + w → ew 频次 6（三个并列第一，取字母序）

语料：`low </w>` | `low e r </w>` | `n ew est</w>` | `w i d est</w>`

这里能看出 BPE 的一个"盲点"：`e+w` 只是 "new" 里的两个字母，并没有语言学含义。BPE 不在乎，它只是在数频率。

ROUND 07 token 总数 48 → 42

候选：`ew+est</w>:6` `n+ew:6` `low+</w>:5` `d+est</w>:3`

ew + est</w> → ewest</w> 频次 6

语料：`low </w>` | `low e r </w>` | `n ewest</w>` | `w i d est</w>`

颗粒度越来越粗：一开始是字母，后来是 2 个字母，现在是 6 个字母。

ROUND 08 token 总数 42 → 36

候选：`n+ewest</w>:6` `low+</w>:5` `d+est</w>:3`

n + ewest</w> → newest</w> 频次 6

语料：`low </w>` | `low e r </w>` | `newest</w>` | `w i d est</w>`

"newest" 因为出现了 6 次，被完整地收进了词表。它是这个语料里的高频词，值得占一个位置。

ROUND 09 token 总数 36 → 31

候选：`low+</w>:5` `d+est</w>:3` `i+d:3` `w+i:3`

low + </w> → low</w> 频次 5

语料：`low</w>` | `low e r </w>` | `newest</w>` | `w i d est</w>`

`low</w>` 表示"独立的 low 这个词"，和 `low`（作为 lower 的前缀）是两个 token。这个设计在真实模型里到处可见。

ROUND 10 token 总数 31 → 28

候选：`d+est</w>:3` `i+d:3` `w+i:3`

d + est</w> → dest</w> 频次 3

语料：`low</w>` | `low e r </w>` | `newest</w>` | `w i dest</w>`

4.4 停下来，看记分牌

10 轮之后，token 总数从 **95 降到 28**。同样的语料，用更少的符号就能表达，这就是"压缩"。





我们一共学到了 10 条规则，**按顺序**如下——这份列表就是模型文件里会保存的东西：

编号	合并	结果	频次
1	e + s	es	9
2	es + t	est	9
3	est + </w>	est</w>	9
4	l + o	lo	7
5	lo + w	low	7
6	e + w	ew	6
7	ew + est</w>	ewest</w>	6
8	n + ewest</w>	newest</w>	6
9	low + </w>	low</w>	5
10	d + est</w>	dest</w>	3

4.5 四个必须自己悟出来的观察

观察一：合并是"滚雪球"的。第 1 轮造出 `es`，第 2 轮才能用 `es` 造 `est`，第 3 轮才能用 `est` 造 `est</w>`。链条一旦断了，后面全废。这就是规则必须有序的物理原因。

观察二：频率在下降。9 → 7 → 6 → 5 → 3。越往后合并的组合越罕见。所以真实训练里，跑到后面几万轮时，每轮只影响极少数位置，收益越来越小。这就是为什么"词表大小"这个超参数存在边际递减。

观察三：它抓到了真实的形态学。`est` 是英语最高级后缀，`er` 是比较级后缀（第 4 章末尾它还没合并，因为只出现 2 次）。算法不懂语法，但它通过"谁和谁老挨在一起"把语法结构摸出来了。

观察四：它也造出了垃圾。`ew`、`dest` 在语言学上毫无意义。BPE 不在乎——只要省 token 就行。这也解释了为什么真实模型的词表里总有一堆看起来莫名其妙的 token。

> **如果你只记住一句话**
>
> BPE 的每一步都是：**找出现次数最多的相邻对，粘起来，记账。**重复到你满意为止。就这么简单。所有复杂性都在工程实现和细节设计上，不在算法思想上。

CHAPTER 05

为什么"挑频率最高的"就对了

> 上一章你在照做，这一章回答"凭什么"。三个视角：压缩、信息、代价。

5.1 压缩视角：它其实在做最优的局部贪心

设某一对 (a, b) 在语料中出现 f 次。合并它之后，这 f 个位置各自从 2 个 token 变成 1 个 token，所以**总 token 数正好减少 f 个。**

那么"这一步能省多少"就等于"这一对的频率"。所以选频率最高的对，等于**每一步都做当前能做的最大压缩。**这是一个标准的贪心算法。

注意：贪心不保证全局最优。有可能先合并一个次高频的对，反而让后面能合并出更大的东西。但 BPE 不在乎——它要的不是最优压缩率，而是"够好的切分方案"。这是个工程取舍，不是数学缺陷。

5.2 信息视角：高频共现 = 一个整体

换个角度想。如果 "t" 和 "h" 在语料里几乎总是一起出现，那说明在语言的真实使用中，它们很少被拆开。它们实际上**行为上像一个单位。**

BPE 做的事情，本质上是**用共现频率来推断"什么算一个整体"。**这个思路在 NLP 里到处都是（n-gram、PMI、词向量），BPE 只是把它用在了"造词表"上。

它当然会犯错。英语里 "the" 出现频率极高，所以 `the` 会成为一个 token——这很对。但 "qu" 也几乎总是一起出现（英语里 q 后面基本必跟 u），于是 `qu` 也成了 token——这对语义没帮助，但它确实减少了一个 token，所以 BPE 照做不误。

> **BPE 不理解语言。**
> **它只是发现：经常挨在一起的东西，大概应该在一起。**

5.3 代价：这个方法的三个"偏见"

偏见一：谁在语料里出现得多，谁就占便宜。如果你用 90% 英文 + 10% 中文的语料训练 BPE，那英文会得到大量整词 token，中文会被切得很碎（因为中文的高频组合没有得到足够的统计支持）。这就是为什么中文模型必须用中文为主的语料训练分词器。

偏见二：罕见词永远被切碎。出现 3 次以下的词，在训练中几乎没有机会被合并成整词。所以人名、专业术语、罕见词在模型眼里永远是碎片拼起来的。这不是 bug，是设计的必然结果。

偏见三：它看不见"边界"以外的东西。因为只看相邻对，BPE 不知道什么是一个"词"。所以预分词（pretokenization）这一步非常关键——先按空格和标点粗切一刀，把"不能跨过去"的边界划出来，BPE 只在词内部合并。第 6 章细讲。

5.4 什么时候停手

词表大小是个纯工程权衡，没有正确答案：

词表偏小	词表偏大
序列变长 → 推理变慢、上下文能装的信息变少 但 embedding 矩阵小、罕见词也能被合理表示	序列变短 → 速度快、上下文效率高 但 embedding 参数暴涨、长尾 token 训练不充分（出现次数太少，学不好）、softmax 变贵

实际的经验区间：多语言模型 10 万～25 万，单语言英文模型 3 万～5 万。GPT-2 选了 50257（因为它要兼顾代码和多语言），LLaMA-3 扩到 128256（为了多语言效率）。这不是理论推导出来的，是试出来的。

> **一个反直觉的事实**
>
> 词表大小一旦定了就不能改。你没法给训练好的模型"加几个新词"——因为 embedding 矩阵的形状固定了。这就是为什么换分词器等于要重训模型，代价极高。也解释了为什么各家的词表大小差异这么大却很少变动。

5.5 算一笔账：词表大小到底花掉多少参数

词表不是免费的。模型第一层是一张 **embedding 表**，形状是 `词表大小 × 隐藏维度`。词表翻倍，这层参数就翻倍。

真实数字（可以自己乘一遍）：

模型	词表 V	隐藏维度 d	Embedding 参数量 = V × d
GPT-2 small	50,257	768	38,597,376 ≈ 3,860 万
LLaMA-3 8B	128,256	4,096	525,336,576 ≈ 5.25 亿

LLaMA-3 8B 总共约 80 亿参数，光 embedding 一层就占了 **6.5%**。如果它当初选了 25 万的词表，这一层会变成 10 亿，占比 12%——多出来的 5 亿参数只换来"少几个 token"，这笔买卖不一定划算。

> **还有一笔隐形成本：长尾 token 学不好**
>
> 词表里排名靠后的那些 token，在整个训练语料里可能只出现几百次。几百次梯度更新，根本不够学出一个靠谱的向量，它们会停在"半随机"的状态。
>
> 所以词表变大的副作用不只是参数变多，还包括**平均每个 token 分到的训练信号变少**。这也是为什么很多模型训练完会发现：罕见 token 的 embedding 范数明显异常，甚至需要专门做"embedding 裁剪"（把过大的异常向量缩回去）。

CHAPTER 06

训练完之后：怎么用它来切句子

> 第 4 章在"造手册"，这一章在"用手册"。两件事的逻辑完全不同，混起来是最常见的错误。

6.1 第一步不是查词表，是预分词

拿到一句新文本，BPE **不会**傻乎乎地从第一个字符开始合并到最后一个。它先要粗切一刀，这叫 **预分词（pretokenization）**。

GPT-2 用的正则表达式是这样的（你可以先跳过细节，只看它切出什么）：

```
pattern = r"'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"
```

它的作用用一句话说：**把句子切成"不可能再跨过去的块"**。具体规则大致是——

- 缩写（'s、't、're…）单独一块
- 连续的字母是一块，**前面的空格跟着走**
- 连续的数字是一块
- 连续的符号是一块
- 连续的空格是一块

切完之后，**BPE 只在每一块内部做合并，绝不跨块**。这个限制极其重要，原因有两个：

原因一：防止标点污染。如果允许跨块合并，"dog." 和 "dog" 会变成完全不同的 token，模型得把这两个东西当成无关的词来学，白白浪费容量。预分词把它们分开，`dog` 就永远是 `dog`。

原因二：防止"合并出一个永不出现的怪物"。如果允许跨词合并，可能会造出 `"cat"+"dog"` 这种组合 token，纯属浪费词表。

> **"前面的空格跟着走"是什么意思**
>
> `"I love cats"` 预分词后是 `["I", " love", " cats"]` —— 注意空格在前。
> 所以 `love` 和 `" love"` 是两个**不同**的 token。这看起来奇怪，但它让模型能区分"句首的词"和"句中的词"，是有意为之的设计。

6.2 编码：严格按 merges 的顺序执行

```
def encode(word, merges):
    toks = list(word) + ["</w>"]          # ① 拆成字符
    for (a, b) in merges:                 # ② 严格按顺序，一步不跳
        out, i = [], 0
        while i < len(toks):
```

```
            if i < len(toks)-1 and toks[i]==a and toks[i+1]==b:
                out.append(a+b); i += 2       # 命中就合并
            else:
                out.append(toks[i]); i += 1
        toks = out
    return toks
```

注意第 ② 行：**这里没有统计，没有比较频率，只是照着列表走一遍。**规则 1 试一次，规则 2 试一次……一直到规则 M。整个过程是 O(M·L)，纯粹的模式替换。

> **这是初学者写错代码的头号原因**
>
> 很多人写编码时，会把 merges 当成一个**集合**去遍历，或者按"哪个能合并就先合并哪个"来写。结果全错。
>
> 正确的模型是：merges 是一个有序脚本，必须从头到尾依次执行。顺序本身携带了全部信息——它记录了训练时"谁先被合并"，也就记录了"谁的优先级更高"。

6.3 拿第 4 章的手册，试三个词

手册就是那 10 条规则。现在用它来编码三个词，看看会发生什么。

① lowest —— 训练时没见过的词

```
初始：l o w e s t </w>
规则1 (e+s)   → l o w es t </w>
规则2 (es+t)  → l o w est </w>
规则3 (est+</w>)→ l o w est</w>
规则4 (l+o)   → lo w est</w>
规则5 (lo+w)  → low est</w>
规则6~10      → 无匹配，跳过
最终：["low", "est</w>"]
```

结果把 "lowest" 拆成了 `low` + `est</w>`。这不是巧合——训练语料里 `low` 和 `est` 都是独立的 token，所以编码器能识别出来。这就是 BPE 的**泛化能力**：没见过的词，也能切成熟人。

② newer —— 训练时也没见过

```
初始：n e w e r </w>
规则遍历一遍，只有规则6 (e+w) 命中：
      n ew e r </w>
其余规则全部无匹配
最终：["n", "ew", "e", "r", "</w>"]   ← 5 个碎片
```

这次切得很碎。因为 "newer" 里的 `er` 在训练语料中只出现 2 次，从没被合并过（第 4 章第 10 轮之后 `e+r` 的频次只有 2，排不上号）。所以编码器只能把它拆成 `e` + `r`。

这就是"罕见词被切碎"的具体样子。不是抽象的说法，是能看见的：5 个 token 换一个词。

③ slow —— 只差一个字母

```
初始：s l o w </w>
规则4 (l+o)      → s lo w </w>
规则5 (lo+w)     → s low </w>
规则9 (low+</w>) → s low</w>      # 命中：low 正好紧跟词尾
其余规则         → 无匹配
最终：["s", "low</w>"]
```

结果是 `s` + `low</w>`。有意思的是：`low</w>` 这个词（作为独立单词的 "low"）在词表里，但 `slow` 不在。所以模型看到 "slow" 时，会理解成"一个 s 加一个 low"。

这解释了一个很多人困惑的现象：为什么模型有时候会把 "slow" 和 "low" 在语义上混淆？因为它们共享了 `low</w>` 这个 token 的 embedding。

6.4 特殊 token

词表里还会塞一些"非文本"的符号，用来给模型发控制信号。它们必须在预分词之前就被识别出来，否则会被拆碎：

Token	作用
<|endoftext|>	文档分隔符，GPT 用它标记"一段文本结束"（也是 padding 的来源）
<s> / </s>	序列开始 / 结束（LLaMA 系列）
<|im_start|>	对话角色边界（ChatML 格式），标记"下面是 user 说的话"
<pad> / <unk>	填充 / 未知（BPE 理论上不需要 unk，但很多实现保留）

CHAPTER 07

字节级 BPE：为什么永远不会"不认识"

> 前面讲的都是"字符级"。现代模型几乎都改成了字节级，这个改动解决了 BPE 最后一个大坑。

7.1 字符级还有个漏洞

字符级 BPE 的基础词表是"所有出现过的字符"。听起来完备，其实不然——**训练语料里没出现过的字符，运行时就是 OOV。**

什么情况会遇到？新出的 emoji（每年 Unicode 都在加）、罕见的汉字（"龘"）、其他语言的字母（阿拉伯文、泰文）、生僻符号（数学符号、音乐符号）。一旦遇到，你就只能吐一个 `<unk>`，信息全丢。

7.2 解法：把文字先变成字节

计算机里所有文字最终都是字节。UTF-8 编码保证：**任何字符都能被编码成 1~4 个字节，而字节只有 256 种可能。**

所以基础词表直接用 256 个字节值当字母表——这就完备了。任何文本，先转成字节序列，再在这个序列上跑 BPE。永远不会有"不在词表里"的东西，因为字节只有 256 种，全都在。



拿实际数据看（这些都是真跑出来的）：

文本	字符数	字节数	UTF-8 字节（十六进制）
hello	5	5	68 65 6c 6c 6f
␣hello	6	6	20 68 65 6c 6c 6f ← 20 就是空格
café	4	5	63 61 66 c3 a9
好	1	3	e5 a5 bd
你好	2	6	e4 bd a0 e5 a5 bd
😀	1	4	f0 9f 99 82

中文的代价，必须知道

英文 1 个字符 = 1 个字节。中文 1 个汉字 = **3 个字节**。所以在字节级 BPE 下，同样的"字符数"，中文要占 3 倍的初始符号位。

虽然训练会把常用汉字合并回单字甚至词（比如 "我们" 可能是一个 token），但**中文 token 效率天生比英文低**。这直接导致：同样的上下文窗口，中文能装的内容更少；同样的 API 价格，中文更贵。这不是模型歧视，是编码方式的物理结果。

（改善办法之一是扩大中文语料占比、扩大词表，让更多汉字和常用词合并成单个 token。国内模型的词表普遍更大，就是这个原因。）

7.3 那些诡异的 `Ġ` 和 `Ċ` 是什么

如果你去看 GPT-2 的 tokenizer 文件，会看到一堆怪符号：`Ġthe`、`Ċ`、`Ã©`。别慌，这有个很土的原因。

BPE 要在字节上操作，但字节值 0~255 里有大量**不可打印**的控制字符（比如 0x00、0x0a）。直接把它们放进 JSON 文件会出各种问题。GPT-2 的解决办法是：**做一个 256 项的双射映射，把每个字节值翻译成一个"看得见、能存进文本"的 Unicode 字符。**

```
字节 0x20（空格） → Ġ      # U+0120
字节 0x0a（换行） → Ċ      # U+010A
字节 0x09（tab）  → ĉ
普通字母保持不变：a→a，b→b ...
```

所以 `Ġthe` 的真实含义就是 `" the"`（前面有个空格）。`Ċ` 就是换行符。看到这些符号时，在脑子里把 `Ġ` 替换成"空格"就通了。

7.4 字节级之后，编码还一样吗

一模一样。唯一的区别是起点：**不是从"字符"开始拆，而是从"字节"开始拆。**

所以第 6 章的 `encode()` 函数只需要改一行：

```python
# 字符级
toks = list(word) + ["</w>"]

# 字节级
toks = list(word.encode("utf-8"))    # 每个元素是 0~255 的整数
```

其余的统计、合并、按序执行，全部不变。这也是为什么说"字节级 BPE 不是新算法，只是换了字母表"。

一句话总结 字符级 BPE 的字母表是"见过的字符"，会漏；字节级的字母表是"全部 256 个字节"，永远不漏。代价是中文这类多字节语言在初始状态下更占位置。

CHAPTER 08

BPE 与 WordPiece、Unigram 的区别

> 这三种方法是子词切分的三条路线。BPE 最笨，但也最稳、最快，所以活得最久。

很多人以为 BERT 用的也是 BPE。**不是。**BERT 用的是 WordPiece，GPT 用 BPE，ALBERT 早期用 Unigram。它们目标相同（切子词），路线完全不同。

	BPE	WordPiece	Unigram
方向	自底向上 从单字符开始，不断合并	自底向上 也是合并，但标准不同	自顶向下 从大词表开始，不断删减
选谁合并	频次最高的相邻对 纯统计，最朴素	使语料**似然提升最大**的对 看"合并后整体概率变好多少"	不合并。算每个子词的概率，删掉最"没用"的
词中出现	不加前缀，靠 `</w>` 标词尾	加 `##` 前缀 `est` → `##est`	SentencePiece 用 `_` 表示空格
编码方式	按 merges 顺序贪心	**最长匹配**（贪心取最长已知子词）	找概率最大的切分（Viterbi）
代表模型	GPT 全系、LLaMA、Qwen、Mistral、RoBERTa	BERT、DistilBERT、MobileBERT	ALBERT、T5、XLNet、mBART
优点	简单、确定、极快、无损	切出来的片段语义更"合理"	有概率模型，支持采样增强
缺点	纯频率，会造出无意义 token	训练慢，实现复杂	训练最慢，编码要做 DP

8.1 WordPiece 到底改了什么

假设有两个候选对：`(a, b)` 出现 10 次，`(c, d)` 出现 8 次。但 `a` 和 `b` 各自都极常见（单独出现几千次），而 `c` 和 `d` 很少单独出现。

BPE 会选 `(a,b)`——因为频率高。

WordPiece 会选 `(c,d)`——因为它算的是 **PMI 式的分数**：`freq(cd) / (freq(c) × freq(d))`。意思是"它们一起出现的频率，比随机碰上的期望高多少倍"。如果一对组合的共现远超随机预期，说明它们**绑定得紧**，值得合并。

> **BPE 问：谁最常见？**
> **WordPiece 问：谁最"离不开彼此"？**

直觉上 WordPiece 更聪明。但实践上差异不大，而 BPE 简单太多，所以在 LLM 时代反而赢了。

8.2 Unigram 是反着来的

Unigram 的思路完全不同：先假设词表里有一大堆候选子词（比如从语料里抽所有高频子串），给每个子词一个概率，然后**不断删掉"删了也不心疼"的子词**，直到词表缩到目标大小。

判断"删了疼不疼"的方法是：用当前的词表，算出语料在最优切分下的总概率（这叫 EM 算法）。删掉某个子词，总概率掉得少，说明它没用，可以删。

最大的好处是它自带**概率分布**，于是可以故意按概率随机采样切分方式（叫 subword regularization），相当于给模型做数据增强。BPE 做不到这一点——它是确定性的，同样输入永远同样输出。

> **为什么 LLM 时代 BPE 赢了**
>
> 因为大模型时代，训练分词器要处理的语料是 TB 级的。BPE 一遍扫描就能出结果，WordPiece 和 Unigram 都要反复迭代。而且 BPE 的编码是纯字符串替换，可以用正则+trie 优化到极致。
>
> **在工程世界里，"够好且快"经常打败"更好但慢"。**

8.3 实际该怎么选

把上面那张表压缩成一个决策清单：

你的情况	选什么
从零训练一个大语言模型	**字节级 BPE**（tiktoken 或 SentencePiece 的 BPE 模式）。快、确定、不会 OOV。
要做中文 / 日文 / 泰文这类无空格语言	**SentencePiece**。它直接吃原始文本，不依赖"按空格切"这个前提。
复用 BERT 生态（分类、NER、检索）	**WordPiece**。直接用现成的 BERT 权重，别自己训。
需要数据增强、或想要"软"切分	**Unigram**。它的 subword regularization 能随机采样切分方式。
只想快速做个小实验	用 `tiktoken` 拿 GPT 的现成词表，一行代码搞定。

> **一个高频误解：SentencePiece 不是算法**
>
> 很多人把 SentencePiece、BPE、WordPiece 并列，其实层级不对。
>
> **BPE 和 Unigram 是算法**（怎么切）；**SentencePiece 是一个工具库**（用什么实现），它内部可以跑 BPE，也可以跑 Unigram。所以 LLaMA 用的是"SentencePiece 里的 BPE 模式"——它既是 BPE，也是 SentencePiece，两句话都对。
>
> 同理，tiktoken 也不是算法，是 OpenAI 写的 BPE 高速实现。

CHAPTER 09

自己写一个：30 行 Python

> 读完不算懂，跑通才算。下面这段代码就是第 4 章那个过程的完整实现，复制粘贴就能用。

9.1 训练

```
from collections import Counter

# ---------- 训练 ----------
def train(corpus, num_merges):
    # ① 初始化：拆成字符 + 词尾标记
    vocab = {tuple(w) + ("</w>",): f for w, f in corpus.items()}
    merges = []

    for _ in range(num_merges):
        # ② 统计所有相邻对（乘上词频）
        pairs = Counter()
        for word, f in vocab.items():
            for i in range(len(word) - 1):
                pairs[(word[i], word[i+1])] += f

        if not pairs:
            break
        # ③ 取频率最高的一对
        best = max(pairs, key=lambda p: (pairs[p], p))
        merges.append(best)

        # ④ 在语料里把这一对全部粘起来
        new_vocab = {}
        for word, f in vocab.items():
            out, i = [], 0
            while i < len(word):
                if i < len(word)-1 and word[i]==best[0] and word[i+1]==best[1]:
                    out.append(best[0] + best[1]); i += 2
                else:
                    out.append(word[i]); i += 1
            new_vocab[tuple(out)] = new_vocab.get(tuple(out), 0) + f
        vocab = new_vocab

    return merges


# ---------- 编码 ----------
def encode(word, merges):
    toks = list(word) + ["</w>"]
    for a, b in merges:          # 顺序不可颠倒
        out, i = [], 0
```

```
        while i < len(toks):
            if i < len(toks)-1 and toks[i]==a and toks[i+1]==b:
                out.append(a+b); i += 2
            else:
                out.append(toks[i]); i += 1
        toks = out
    return toks


# ---------- 跑起来 ----------
corpus = {"low": 5, "lower": 2, "newest": 6, "widest": 3}
m = train(corpus, 10)
print(m)
print(encode("lowest", m))   # ['low', 'est</w>']
print(encode("newer",  m))   # ['n', 'ew', 'e', 'r', '</w>']
print(encode("slow",   m))   # ['s', 'low</w>']
```

9.2 代码里三个值得注意的细节

细节一：`vocab` 的 key 是 tuple，不是 string。因为一个词被拆成多个 token，必须用元组保存序列。用字符串会导致 "low" 和 "l"+"ow" 混淆。

细节二：`max(pairs, key=lambda p: (pairs[p], p))`。这里用了元组比较：先比频率，频率相同就比字母序（保证结果确定，不会因为字典顺序变化而变）。第 4 章那些"三个并列第一取字母序"就是这段代码干的。

细节三：`new_vocab[tuple(out)] = new_vocab.get(...) + f`。合并后不同词可能变成同一个序列，比如 "low" 和 "low" 合并后还是同一个，所以要累加频次而不是覆盖。

9.3 这段代码缺什么

它能跑通原理，但离生产可用还差得远。真实的 tokenizer 还要处理：

- **预分词**：按空格、标点、数字粗切，不能只用 `corpus` 里的整词（第 6 章）
- **字节编码**：把 UTF-8 字节当字母表，而不是字符（第 7 章）
- **增量更新**：不能每轮重扫全语料，要只更新受影响位置（第 3 章）
- **特殊 token**：`<|endoftext|>` 之类要单独处理，不能被拆开
- **速度**：真实实现用 C++/Rust，配合正则和 trie，比纯 Python 快几百倍

想直接上手真实的？装 `tokenizers` 库，五行代码：

```
from tokenizers import Tokenizer, models, trainers, pre_tokenizers

tk = Tokenizer(models.BPE())
tk.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space=False)
trainer = trainers.BpeTrainer(vocab_size=30000, special_tokens=["<|endoftext|>"])
tk.train_from_iterator(open("corpus.txt", encoding="utf-8"), trainer)
```

CHAPTER 10

十个常见误解

> 这些都是我在实际教学里被问过很多次的问题。每条都直给答案。

---

1 "BPE 切出来的一定是有意义的词素吧？"

不保证。看第 4 章的 `ew` 和 `dest`，纯属噪声。BPE 只优化"省多少 token"，不优化"有没有语言学意义"。它在统计上偶然捕捉到了形态学，但不是设计目标。真实模型的词表里大量 token 是跨词素的碎片。

2 "空格无关紧要，预处理去掉就行。"

大错。 `hello` 和 `" hello"`（前导空格）是两个完全不同的 token 序列。去掉空格会让 "the cat" 和 "thecat" 变成一样的东西，模型彻底失去词边界信息。GPT-2 用 `Ġ` 显式编码空格，就是这个原因。

3 "词表越大，模型越强。"

不一定。词表大 → 序列短 → 同样上下文能装更多内容，这是好处。但 embedding 矩阵是 `vocab_size × hidden_dim`，词表翻倍，这部分参数翻倍；而且长尾 token 出现次数太少，训练不充分，embedding 学得很差。**这是个 U 形曲线，不是单调的。**

4 "为什么 GPT 数不清 strawberry 里有几个 r？"

因为 `strawberry` 被切成了 `straw` + `berry` 两个 token（可能还更碎）。模型看到的是"两个整块的 embedding"，它**看不到里面的字母**。你让它数 r，相当于让人看两个汉字问"这里面有几笔"。这是分词粒度和字符级任务之间的根本矛盾，不是模型笨。

5 "合并规则就是一个集合，顺序无所谓。"

顺序就是一切。第 2 条规则依赖第 1 条的产物。如果打乱顺序，`es+t` 这一条会找不到 `es`，直接失效。你存 tokenizer 时存的是**有序列表**，不是集合。

6 "编码的时候要重新统计频率。"

绝对不用。编码只是照着 merges 列表顺序做字符串替换。统计频率是训练阶段的事，发生一次，之后就固化了。如果编码要统计频率，那每处理一句话都得扫一遍语料，根本不现实。

7 "中文一个字就是一个 token。"

不一定，通常不是。取决于词表和语料。常见汉字可能是一个 token，罕见字会被拆成 2~3 个字节级 token。同一个汉字在不同上下文里也可能切法不同（因为在词内部和外部的组合不一样）。

8 "分词器可以换，换个更好的就行。"

换分词器 = 重训模型。因为 token ID 和 embedding 矩阵是绑定的。换了分词器，所有 ID 的含义都变了，原来的权重全部作废。这是模型架构里最"绑死"的组件之一。

9 "BPE 是可逆的，切完一定能拼回去。"

BPE 本身是无损的，token 拼起来就是原文。但要注意两点：① 如果把特殊情况（如 `<|endoftext|>`）当普通文本处理，可能出问题；② 有些实现会做 Unicode 规范化（NFKC），这会改变原文（比如全角转半角）。**算法无损，工程上不一定。**

10 "token 就是"词"，tokenize 就是分词。"

中文叫"分词"，但 BPE 做的不是语言学的分词。它切出来的东西可能是一个词、半个词、两个词的一半、或者纯粹的音节碎片。更准确的说法是 **subword tokenization**——子词切分。"token" 翻译成"词元"或"标记"都比"词"准确。

> **检验你是不是真懂了**
>
> 下面这个问题能答对，说明你过关了：
>
> 训练语料里 "cat" 出现 10000 次，"cats" 出现 9000 次。"cat" 会不会成为一个 token？"cats" 呢？
>
> 答案："cat" 几乎肯定会（高频且是 "cats" 的前缀，会先被合并出来）；"cats" 也会，但要看词表大小——如果词表够大就会被合并，否则会留在 `cat` + `s`。而 `cat` 这个 token 的存在，意味着模型处理 "cats" 时是"认识 cat，再加个 s"，而不是把它当一个全新词。

APPENDIX A

术语表

> 按出现顺序排列。忘了哪个词，回来查。

术语	人话解释
Token	模型处理的最小单位。可能是一个字、一个词、半个词、几个字节。
Vocabulary（词表）	所有合法 token 的集合，每个有唯一编号。大小一般在 3 万～25 万。
BPE	Byte Pair Encoding。反复合并最高频相邻对的算法。1994 年为压缩发明，2016 年用于分词。
Merge（合并）	把两个相邻 token 粘成一个新 token，并记录到规则表。
Merges（合并规则）	训练产出的**有序**列表。顺序 = 优先级，不可打乱。
Subword（子词）	介于"字符"和"词"之间的切分单位。BPE 的产物。
OOV	Out-of-Vocabulary，词表外的词。字节级 BPE 下不存在这个问题。
Pre-tokenization	预分词。先用正则粗切（按空格/标点/数字），BPE 只在块内合并。
</w>	词尾标记。区分"词尾后缀"和"词中片段"。
Ġ / Ċ	GPT-2 的字节可视化。Ġ = 空格，Ċ = 换行。
Byte-level BPE	以 UTF-8 字节（256 个）为字母表的 BPE。永远不会遇到 OOV。
WordPiece	BERT 用的方法。按"似然提升"选合并对象，编码用最长匹配。
Unigram	T5/ALBERT 用的方法。从大词表剪枝，自带概率，支持采样。
SentencePiece	Google 的分词工具库，支持 BPE 和 Unigram，直接处理原始文本（不依赖预分词）。
tiktoken	OpenAI 的快速 BPE 实现，GPT-3.5/4 用的。

附录 B • 一页速查

训练（一次性）

① 统计语料 → ② 全部拆成字符（+词尾标记）→ ③ 数相邻对 → ④ 合并最高频的 → ⑤ 记进 merges → 重复 ③④⑤ 直到词表达标

产出：词表 + merges 有序列表。

编码（每次输入）

① 预分词（按空格/标点粗切）→ ② 每块拆成字符（或字节）→ ③ **按 merges 顺序**逐条尝试合并 → ④ 查表转 ID

注意：不统计、不比较、不做任何学习。

三个核心直觉

① **压缩。**合并一对频率为 f 的组合，token 总数正好减 f。选最高频 = 当前最优压缩。

② **共现即整体。**老挨在一起的东西，大概应该在一起。统计频率间接捕捉了形态学。

③ **顺序即知识。**merges 的顺序记录了训练时的合并先后，它携带了全部信息。打乱即失效。

如果只能记一句话

BPE 是一个只会数数的算法：它反复找出语料里最常挨在一起的那两个符号，把它们粘成一个新符号。
反复几万次之后，它粘出来的东西，看起来像语言的骨架。

→ 读完这一页之后，可以做三件事

一、去亲眼看一次切分结果。随便找一个在线的 tokenizer 可视化页面，把同一句话分别用中文和英文输入，看它被切成几块、每块对应的 ID 是什么。你会立刻对"中文更费 token"有体感。

二、自己训一个。把第 9 章的语料换成几十万字的真实中文，跑 1000 轮，然后打印词表。你会看到"我们""什么""可以"这类高频组合自己长出来——那一刻比读十篇文章都管用。

三、读一个真实的词表。下载 `cl100k_base` 或任意开源模型的 `tokenizer.json`，找出里面最长的 token 和最短的 token。你会看到一堆残缺的单词碎片，然后真正理解"它只是统计，不是理解"。

本文所有演算数据由配套脚本真实运行生成，可复现。语料取自 Sennrich et al. (2016) 的经典示例。章节中的数值（95 → 28 个 token、10 条合并规则、各轮频次）均为程序输出，未做人工修饰。